

Healthcare Patient Analytics

A Healthcare Data Analysis Project

Tools & Technologies

Excel • PostgreSQL • Power BI • SQL

Author

Alejandro Llavona Maldonado

2026

Project Overview

This project consists of analysis performed on a healthcare dataset of 55,000+ patient records using PostgreSQL to clean and query data, then building an interactive Power BI dashboard to identify trends in hospital admissions, billing costs, and patient demographics."

The project is structured in seven key steps:

Phase	Name	Description
1	Download a Dataset	Utilize a free healthcare dataset sourced from Kaggle, a prominent online platform and community for data science and machine learning,
2	Set up a Database	After sourcing a proper dataset, utilize PostgreSQL to create a database for our data.
3	Create Tables and Import Data	This step involves utilizing SQL to create the table that will store the data inside the database, then importing it from the .CSV files.
4	Clean the Data	Write a series of SQL queries to clean the data (Check for NULLS, Duplicates, etc.)
5	Analyze the Data	Write a series of SQL queries to answer key questions that present valuable insights regarding the data.
6	Connect to Power BI	Establish a connection from the PostgreSQL database and Microsoft Power BI to transfer the data and present it in Power BI.
7	Design the Dashboard	Familiarize myself with the layout of Power BI and create an interactive dashboard that utilizes different visuals to present the data.

The central analytical purpose of this project:

| "Identify trends in hospital admissions, billing costs, and patient demographics. "

Step 1: Download a Dataset

I headed to Kaggle.com to source a dataset regarding healthcare. For this project, I used the free “Healthcare Dataset – Dummy data with Multi Category Classification Problem” by Prasad Patil.

Then simply just placed the .CSV file into my dedicated project folder, utilizing Excel to verify the data was there and get an initial preview of what I would be working with.

Step 2: Setup a Database

After successfully acquiring the dataset, I utilized PostgreSQL through *pgAdmin4* to create my database through the following steps:

- Servers -> PostgreSQL 18 -> Databases -> Right Click -> Create -> Database

I named it *healthcare_project*.

Step 3: Create Tables and Import Data

Part A — Create the Table

I used the following query to create the table and let it know which columns are coming in. This matches the ones in the .csv / excel file:

```
CREATE TABLE patients (  
    name VARCHAR(100),  
    age INT,  
    gender VARCHAR(10),  
    blood_type VARCHAR(5),  
    medical_condition VARCHAR(100),  
    date_of_admission DATE,  
    doctor VARCHAR(100),  
    hospital VARCHAR(100),  
    insurance_provider VARCHAR(100),  
    billing_amount NUMERIC(10,2),  
    room_number INT,  
    admission_type VARCHAR(50),  
    discharge_date DATE,  
    medication VARCHAR(100),  
    test_results VARCHAR(50)  
);
```

Part B — Import the CSV

1. In the left Browser panel, I expanded *healthcare_project* → *Schemas* → *public* → *Tables*
2. My *patients* table was listed there
3. Right-click on patients → click Import/Export Data...
4. A window popped up — I set it up like this:
 - Toggle at the top: set to Import
 - Filename: clicked the folder icon and navigated to my Healthcare_BA_Project folder and selected healthcare_dataset.csv
 - Format: make sure it says csv
5. Clicked the Options tab at the top of that same window and make sure:
 - Header is toggled ON — this tells PostgreSQL the first row is column names, not actual data
 - Delimiter is set to , (a comma)

Part C — Verify the data loaded correctly

In the Query Tool, ran this:

```
SELECT * FROM patients LIMIT 10;
```

Step 4: Clean the Data

1. Ran the following query to check how many rows I had

```
SELECT COUNT(*) FROM patients;
```

This successfully returned 55,500

2. Check for NULL values with the following query.

```
SELECT  
COUNT(*) FILTER (WHERE billing_amount IS NULL) AS missing_billing,  
COUNT(*) FILTER (WHERE age IS NULL) AS missing_age,  
COUNT(*) FILTER (WHERE medical_condition IS NULL) AS missing_condition  
FROM patients;
```

3. Check for duplicate rows with the following query:

```
SELECT name, date_of_admission, COUNT(*)  
FROM patients  
GROUP BY name, date_of_admission  
HAVING COUNT(*) > 1;
```

In this query, I utilized **HAVING** as it filters groups after grouping, different from **WHERE**, which does it before grouping.

It showed that 5500 rows were duplicated, I checked the first one just to see with this query:

```
SELECT * FROM patients  
WHERE name = 'MaKayla CONRAD'  
AND date_of_admission = '2020-06-22';
```

After analyzing, everything was identical except the age (56 and 58). The chances of this are close to zero considering all the possibilities (name, dr, condition, blood type, etc). So, I ran the following command to delete all duplicates.

```
DELETE FROM patients  
WHERE ctid NOT IN (  
SELECT MIN(ctid)  
FROM patients  
GROUP BY name, date_of_admission, medical_condition, hospital  
);
```

This is essentially a 2-part query

First, the inner query:

- **ctid**: unique row ID, like a fingerprint for each row
- **GROUP BY name, date_of_admission, medical_condition, hospital**: groups rows that share all four of those values together (the duplicates)
- **MIN(ctid)**: picks just one row from each group to keep — specifically the one with the lowest ctid (i.e. the first one that was entered)

The result is a list of ctid values — one per group — representing the rows we want to keep. Then the outer query simply deletes rows that aren't in that list we created.

4. Back in Step 1, I remember seeing in Excel that the names weren't capitalized properly, so I ran this simple query to fix that

```
UPDATE patients
SET name = INITCAP(name);
```

5. Lastly, I checked out the data at a glance with this query:

```
SELECT COUNT(*) AS total_patients,
       MIN(age) AS youngest_patient,
       MAX(age) AS oldest_patient,
       COUNT(DISTINCT medical_condition) AS unique_conditions,
       COUNT(DISTINCT hospital) AS unique_hospitals
FROM patients;
```

COUNT(*) AS total_patients **COUNT(*)**: This uses **total_patients** as an alias to tell me the amount of rows we have, which is equal to how many patients we have.

MIN(age) AS youngest_patient **MIN()**: returns the lowest value in the column as the youngest patient, in this case **18**.

MAX(age) AS oldest_patient: the opposite of **MIN()**, finds the largest value. **MAX(age)** returned **85**.

COUNT(DISTINCT medical_condition) AS unique_conditions **DISTINCT** ignores repeats, so it returns each medical condition only once, giving us the amount of unique conditions, **6** in our case.

Step 5: Analyze the Data

The primary objective during this step was to think of questions regarding what trends or insights I would like to know about this data set, then create SQL queries that could answer these questions. The results for all the queries I saved in .txt files, found in the project folder

1. What are the most common medical conditions?

```
SELECT medical_condition,
COUNT(*) AS patient_count
FROM patients
GROUP BY medical_condition
ORDER BY patient_count DESC;
```

2. What is the average billing amount by condition?

```
SELECT medical_condition,
ROUND(AVG(billing_amount), 2) AS avg_billing
FROM patients
GROUP BY medical_condition
ORDER BY avg_billing DESC;
```

3. How many patients per admission type?

```
SELECT admission_type,
COUNT(*) AS total_patients,
ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER(), 1) AS percentage
FROM patients
GROUP BY admission_type
ORDER BY total_patients DESC;
```

This query implements the **OVER() window function**.

- Without OVER(), SUM(COUNT(*)) would only add up within each group
- Adding OVER() tells SQL "calculate this across the **entire** table, not just the current group
- So, $100.0 * \text{COUNT}(*)/\text{SUM}(\text{COUNT}(*)) \text{ OVER}()$ gives you each admission type as a **percentage of all patients**
- 100.0 (not just 100) forces SQL to use decimal math instead of rounding to whole numbers

4. What is the average billing and patient type by Insurance Provider?

```
SELECT insurance_provider,  
COUNT(*) AS total_patients,  
ROUND(AVG(billing_amount), 2) AS avg_billing,  
ROUND(MIN(billing_amount), 2) AS min_billing,  
ROUND(MAX(billing_amount), 2) AS max_billing  
FROM patients  
GROUP BY insurance_provider  
ORDER BY avg_billing DESC;
```

Using multiple aggregate functions — COUNT, AVG, MIN, and MAX — in one query efficiently gets me a full financial picture per insurance provider all at once.

During this step I noticed something in the data. I saw that in the min_billing column, I had some negative values. This caused me to believe that either

- A. The data set includes refunds or adjustments, or
- B. there was a data entry error.

So, I ran the following query:

```
SELECT name, medical_condition, admission_type, insurance_provider,  
billing_amount  
FROM patients  
WHERE billing_amount < 0;
```

This showed me all the negative billing amounts, and there were too many for it to be a data entry error, since that would only consist of smaller amount, yet I saw 96 results, a realistic amount for the dataset consisting of 50k (55500k originally)

5. What is the average length of hospital stay by medical condition?

```
SELECT medical_condition,  
ROUND(AVG(discharge_date - date_of_admission), 1) AS avg_stay_days  
FROM patients  
GROUP BY medical_condition  
ORDER BY avg_stay_days DESC;
```

In PostgreSQL you can subtract two date columns from each other and it automatically gives you the number of days between them.

6. What are the demographics by age?

```
SELECT  
CASE  
    WHEN age < 18 THEN 'Under 18'  
    WHEN age BETWEEN 18 AND 35 THEN '18-35'  
    WHEN age BETWEEN 36 AND 55 THEN '36-55'  
    WHEN age BETWEEN 56 AND 70 THEN '56-70'  
    ELSE '70+'  
END AS age_group,
```

```
COUNT(*) AS patient_count
FROM patients
GROUP BY age_group
ORDER BY patient_count DESC;
```

CASE WHEN works exactly like an IF statement in Excel.

7. How many test results of each type were there?

```
SELECT test_results,
       COUNT(*) AS total,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER(), 1) AS percentage
FROM patients
GROUP BY test_results
ORDER BY total DESC;
```

Step 6: Connect to Power BI

In a blank report in *Power BI*, I selected *Home* -> *Get Data* -> *More* -> *PostgreSQL Database*

I then selected the server as localhost (database is installed locally), then my database, *healthcare_project*.

After entering my username and password (master password when installing PostgreSQL). I utilized the navigator to initially view my table with all the data, then utilized "Table View" to confirm all 50k data sets loaded successfully.

Step 7: Design the Dashboard

This final step involved presenting the insights I found regarding my data in a meaningful way. For this, I used **6 key visuals**

Patient Count by Medical Condition: Utilizing a Clustered Bar Chart, with the *medical_condition* as the Y-axis, and the count of *name* as the X-axis.

Average Billing Amount by Condition: Utilizing a Clustered Column Chart, with the *medical_condition* as the x-axis, and the Average of *billing_amount* for the Y-axis

Admissions by Type: Utilizing a Donut Chart with *admission_type* as the legend, and count of *name* for the values.

Patient Count by Age Group: Utilizing a Clustered Bar Chart, with count of *name* for the X-axis, and a new column created via DAX formula named *Age Group* for the Y-axis:

```
Age Group =
SWITCH(
    TRUE(),
    'public patients'[age] < 18, "Under 18",
    'public patients'[age] <= 35, "18-35",
    'public patients'[age] <= 55, "36-55",
    'public patients'[age] <= 70, "56-70",
    "70+"
)
```

This essentially serves as a CASE WHEN that labels ages by age group depending on the case, with a "70+" serving as an ELSE.

Average Days of Stay by Medical Condition: Utilizing a Treemap with *medical_condition* for the categories, and a new DAX measure named *Avg Stay Days* for the values:

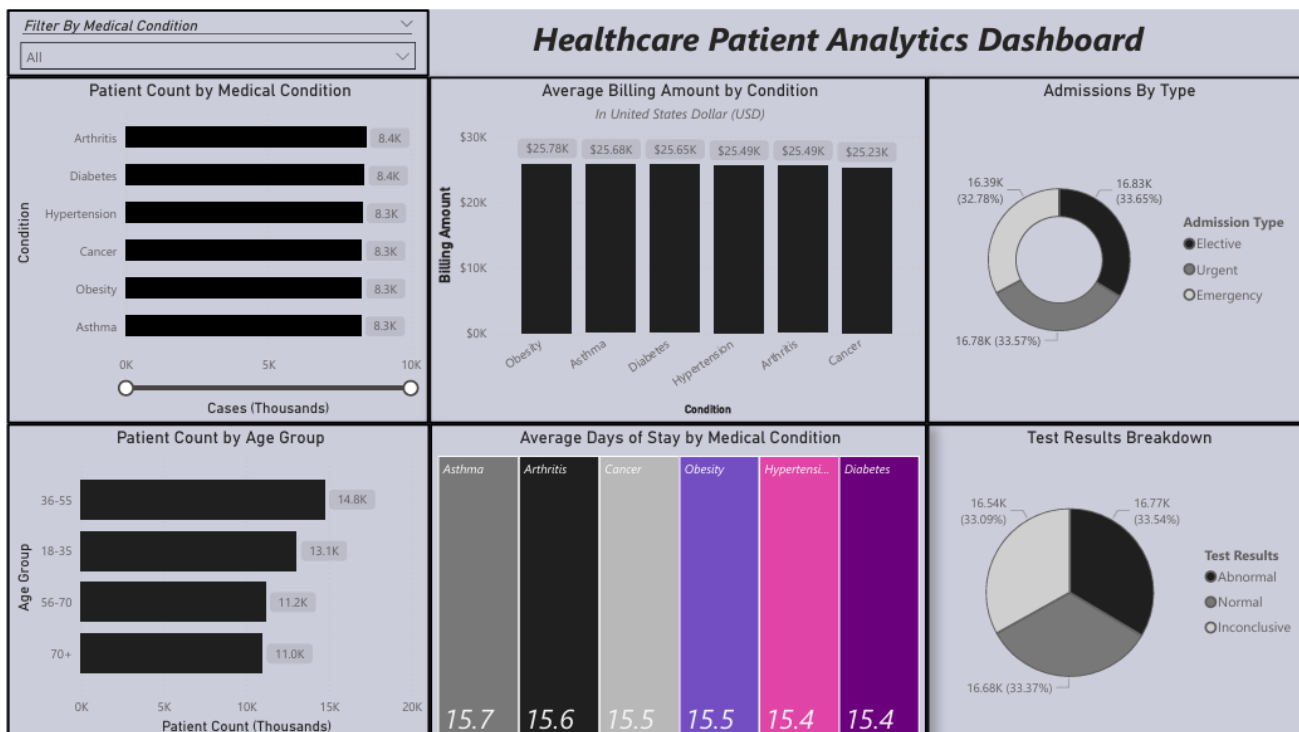
```
Avg Stay Days =
AVERAGEX (
    'public_patients',
    DATEDIFF (
        'public_patients'[date_of_admission],
        'public_patients'[discharge_date],
        DAY
    )
)
```

This differs from the last DAX *formula*, as it **does not create a new table**, it serves as a **calculation that's computed dynamically**.

- **DATEDIFF()** calculates the difference between two dates. its parameters are the start date, the end date, and the unit you want (DAY in this case)
- **AVERAGEX()**: instead of averaging a single column directly (AVERAGE()), it goes through each row one by one, calculates the DATEDIFF for that row, then averages all those results together

Test Results Breakdown: Utilizing a Pie Chart with *test_results* for the legend, and count of *name* for the values

Finally, I included a **Slicer** that serves as an interactive *filter by medical conditions* button.



References

- Kaggle.com: <https://www.kaggle.com/datasets/prasad22/healthcare-dataset>
- *PostgreSQL 18.3 Documentation*. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/>
- *W3Schools.com*. SQL Tutorial. <https://www.w3schools.com/sql/default.asp>
- *Microsoft Data Analysis Expressions (DAX) reference - DAX*. Microsoft Learn. <https://learn.microsoft.com/en-us/dax/>